LAB 3

1.SJF Non Preemptive

```
#include<stdio.h>
#include <limits.h>
struct process{
int pid;     int at;
int bt;     int ft;
int ct;     int tat;
int wt;     int rt;
   int completed;
};

int main(){     int
n,i,time=0,completed=0,idx;     struct
process p[20];     float
avg_wt=0,avg_tat=0,avg_rt=0;
printf("Enter number of processes: ");
scanf("%d",&n);

   printf("\nEnter Arrival Time and Burst Time:\n");
for(i=0;i<n;i++)
   {        p[i].pid=i+1;
printf("P%d AT: ",p[i].pid);
scanf("%d",&p[i].at);
printf("P%d BT: ",p[i].pid);
scanf("%d",&p[i].bt);
p[i].completed=0;
   }

   while(completed<n)
   {
      int min_bt=INT_MAX;
      idx=-1;
for(i=0;i<n;i++)
      {
         if(p[i].at<=time && p[i].completed==0)
```

```
        {               if(p[i].bt
< min_bt)
            {
min_bt=p[i].bt;
idx=i;
            }
            if(p[i].bt==min_bt)
            {                   if(p[i].at
< p[idx].at)                    idx=i;
            }
        }       }       if(idx!=-1)
{           p[idx].ft=time;
p[idx].ct=time+p[idx].bt;
time=p[idx].ct;

        p[idx].tat=p[idx].ct-p[idx].at;
p[idx].wt=p[idx].tat-p[idx].bt;         p[idx].rt=p[idx].ft-
p[idx].at;

        avg_wt+=p[idx].wt;
avg_tat+=p[idx].tat;            avg_rt+=p[idx].rt;

        p[idx].completed=1;
completed++;
      }
else        {
time++;
      }
   }
   printf("\nPID\tAT\tBT\tFT\tCT\tTAT\tWT\tRT\n");

   for(i=0;i<n;i++)
   {
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
      p[i].pid,p[i].at,p[i].bt,p[i].ft,p[i].ct,p[i].tat,p[i].wt,p[i].rt);
}
```

```
    printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f",avg_rt/n);

    return 0; }
```

Output

```
C:\Users\student\Downloads\

Enter number of processes: 5

Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 3
P2 AT: 2
P2 BT: 2
P3 AT: 3
P3 BT: 5
P4 AT: 4
P4 BT: 4
P5 AT: 6
P5 BT: 1

PID     AT      BT      FT      CT      TAT     WT      RT
P1      0       3       0       3       3       0       0
P2      2       2       3       5       3       1       1
P3      3       5       10      15      12      7       7
P4      4       4       5       9       5       1       1
P5      6       1       9       10      4       3       3

Average Waiting Time = 2.40
Average Turnaround Time = 5.40
Average Response Time = 2.40
Process returned 0 (0x0)   execution time : 15.825 s
Press any key to continue.
```

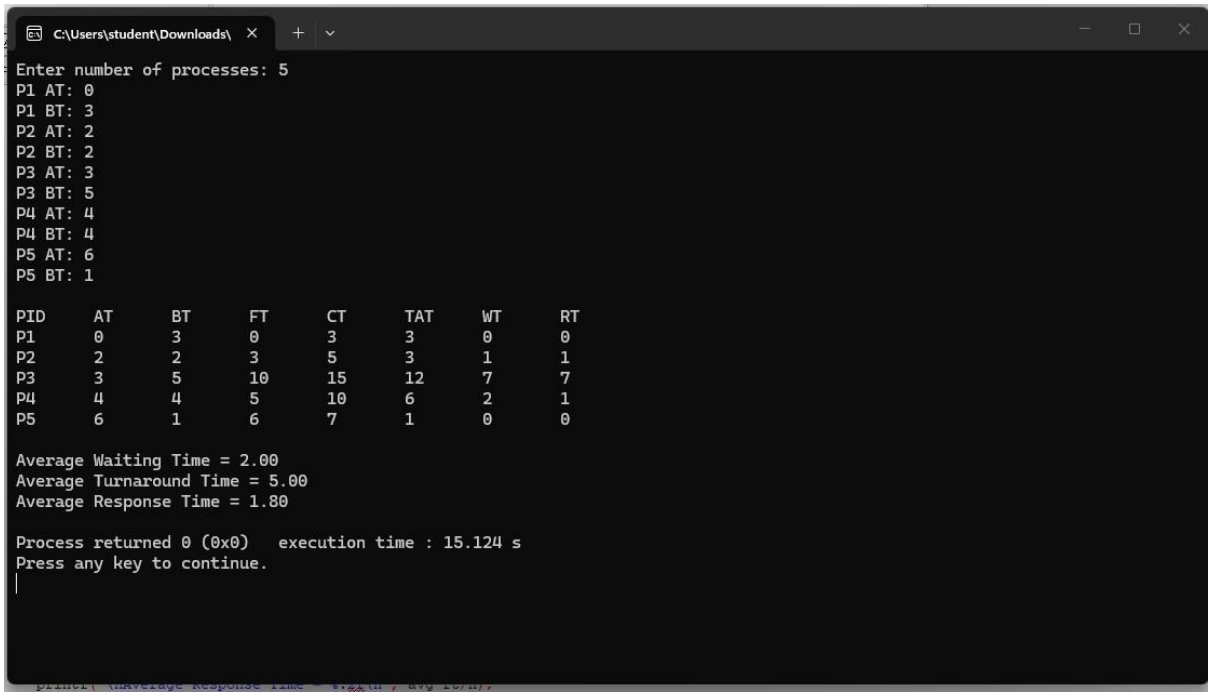
2. SJF Preemptive

```
#include <stdio.h>
#include <limits.h>

struct process {
int pid;     int
at;     int bt;
int rem_bt;
int ct;     int ft;
int wt;     int
tat;     int rt;
int started;
};
```

```
int main() {     int n, i, completed = 0,
time = 0, idx;     float avg_wt=0,
avg_tat=0, avg_rt=0;

   printf("Enter number of processes: ");
scanf("%d",&n);

   struct process p[20];
for(i=0;i<n;i++) {         p[i].pid =
i+1;         printf("P%d AT: ",
p[i].pid);         scanf("%d",
&p[i].at);         printf("P%d BT:
", p[i].pid);         scanf("%d",
&p[i].bt);         p[i].rem_bt =
p[i].bt;         p[i].started = 0;
   }

   while(completed < n) {
int min_rem = INT_MAX;
       idx = -1;



       for(i=0;i<n;i++) {             if(p[i].at <=
time && p[i].rem_bt > 0) {
if(p[i].rem_bt < min_rem) {
min_rem = p[i].rem_bt;                    idx = i;
               }

               if(p[i].rem_bt == min_rem) {
if(p[i].at < p[idx].at)                    idx
= i;
               }
           }
       }

       if(idx != -1) {
```

```
            if(p[idx].started == 0) {
p[idx].ft = time;                p[idx].rt =
p[idx].ft - p[idx].at;
p[idx].started = 1;
            }

            p[idx].rem_bt--;
time++;

            if(p[idx].rem_bt == 0) {
p[idx].ct = time;                p[idx].tat =
p[idx].ct - p[idx].at;                p[idx].wt =
p[idx].tat - p[idx].bt;

                avg_wt += p[idx].wt;
avg_tat += p[idx].tat;                avg_rt
+= p[idx].rt;

                completed++;
            }
        } else {

            time++;
        }
    }
    printf("\nPID\tAT\tBT\tFT\tCT\tTAT\tWT\tRT\n");
for(i=0;i<n;i++) {
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].ft, p[i].ct,
p[i].tat, p[i].wt, p[i].rt);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt/n);
printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
printf("\nAverage Response Time = %.2f\n", avg_rt/n);

    return 0; }
```

Output



```
C:\Users\student\Downloads\

Enter number of processes: 5
P1 AT: 0
P1 BT: 3
P2 AT: 2
P2 BT: 2
P3 AT: 3
P3 BT: 5
P4 AT: 4
P4 BT: 4
P5 AT: 6
P5 BT: 1

PID     AT      BT      FT      CT      TAT     WT      RT
P1      0       3       0       3       3       0       0
P2      2       2       3       5       3       1       1
P3      3       5       10      15      12      7       7
P4      4       4       5       10      6       2       1
P5      6       1       6       7       1       0       0

Average Waiting Time = 2.00
Average Turnaround Time = 5.00
Average Response Time = 1.80

Process returned 0 (0x0)   execution time : 15.124 s
Press any key to continue.
```

3. Priority Non Preemptive

```
#include <stdio.h>
#include <limits.h>

struct process {
int pid;     int
at;     int bt;
int priority;
int ct;     int ft;
int wt;     int
tat;     int rt;
   int completed;
};

int main() {     int n, i, completed = 0, time
= 0, idx;     float avg_wt = 0, avg_tat = 0,
avg_rt = 0;

   printf("Enter number of processes: ");
scanf("%d", &n);
```

```
    struct process p[20];

    printf("Enter Arrival Time, Burst Time and Priority (lower value = higher
priority):\n");     for (i = 0; i < n; i++) {         p[i].pid = i + 1;
printf("P%d AT: ", p[i].pid);         scanf("%d", &p[i].at);         printf("P%d
BT: ", p[i].pid);         scanf("%d", &p[i].bt);         printf("P%d Priority: ",
p[i].pid);         scanf("%d", &p[i].priority);         p[i].completed = 0;
    }

    while (completed < n) {         int
highest_priority = INT_MAX;
idx = -1;

        for (i = 0; i < n; i++) {             if (p[i].at <=
time && p[i].completed == 0) {                 if
(p[i].priority < highest_priority) {
highest_priority = p[i].priority;                     idx = i;
                }
                if (p[i].priority == highest_priority) {
if (p[i].at < p[idx].at)                     idx = i;
                }
            }
        }

        if (idx != -1) {             p[idx].ft =
time;             p[idx].rt = p[idx].ft -
p[idx].at;

            time += p[idx].bt;
p[idx].ct = time;

            p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;

            avg_wt += p[idx].wt;
avg_tat += p[idx].tat;             avg_rt
+= p[idx].rt;
```

```
            p[idx].completed = 1;
completed++;        } else {
time++;
        }
    }

    printf("\nPID\tAT\tBT\tPriority\tFT\tCT\tTAT\tWT\tRT\n");
for (i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\t%d\t%d\n",
                p[i].pid, p[i].at, p[i].bt, p[i].priority,
p[i].ft, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f", avg_tat / n);
printf("\nAverage Response Time = %.2f\n", avg_rt / n);

    return 0; }
```

Output

```
C:\Users\student\Downloads\

Enter number of processes: 5
Enter Arrival Time, Burst Time and Priority (lower value = higher priority):
P1 AT: 0
P1 BT: 3
P1 Priority: 5
P2 AT: 2
P2 BT: 2
P2 Priority: 3
P3 AT: 3
P3 BT: 4
P3 Priority: 2
P4 AT: 4
P4 BT: 4
P4 Priority: 4
P5 AT: 6
P5 BT: 1
P5 Priority: 1

PID     AT      BT      Priority        FT      CT      TAT     WT      RT
P1      0       3       5               0       3       3       0       0
P2      2       2       3               8       10      8       6       6
P3      3       4       2               3       7       4       0       0
P4      4       4       4               10      14      10      6       6
P5      6       1       1               7       8       2       1       1

Average Waiting Time = 2.60
Average Turnaround Time = 5.40
Average Response Time = 2.60

Process returned 0 (0x0)   execution time : 57.824 s
Press any key to continue.
```

4. Priority Preemptive

```
#include <stdio.h>
#include <limits.h>

struct process {
int pid;     int
at;     int bt;
int rem_bt;
int priority;
int ct;     int ft;
int wt;     int
tat;     int rt;
int started;
};

int main() {     int n, i, completed = 0, time
= 0, idx;     float avg_wt = 0, avg_tat = 0,
avg_rt = 0;

   printf("Enter number of processes: ");
scanf("%d", &n);

   struct process p[20];

   printf("Enter Arrival Time, Burst Time and Priority (lower value = higher
priority):\n");     for (i = 0; i < n; i++) {         p[i].pid = i + 1;
printf("P%d AT: ", p[i].pid);         scanf("%d", &p[i].at);         printf("P%d
BT: ", p[i].pid);         scanf("%d", &p[i].bt);         printf("P%d Priority: ",
p[i].pid);         scanf("%d", &p[i].priority);         p[i].rem_bt = p[i].bt;
       p[i].started = 0;
   }

   while (completed < n) {         int
highest_priority = INT_MAX;
idx = -1;

       for (i = 0; i < n; i++) {             if (p[i].at <=
time && p[i].rem_bt > 0) {                 if
(p[i].priority < highest_priority) {
```

```
highest_priority = p[i].priority;                    idx
= i;
                }
                if (p[i].priority == highest_priority) {
if (p[i].at < p[idx].at)                        idx = i;
                }
            }
        }

        if (idx != -1) {            if
(p[idx].started == 0) {
p[idx].ft = time;                p[idx].rt =
p[idx].ft - p[idx].at;
p[idx].started = 1;
            }

            p[idx].rem_bt--;
time++;

            if (p[idx].rem_bt == 0) {
p[idx].ct = time;                p[idx].tat =
p[idx].ct - p[idx].at;                p[idx].wt =
p[idx].tat - p[idx].bt;

                avg_wt += p[idx].wt;
avg_tat += p[idx].tat;                avg_rt
+= p[idx].rt;

                completed++;
            }
        } else {
time++;
        }
    }

    printf("\nPID\tAT\tBT\tPriority\tFT\tCT\tTAT\tWT\tRT\n");
for (i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\t%d\t%d\n",
```

```
            p[i].pid, p[i].at, p[i].bt, p[i].priority,
p[i].ft, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f", avg_tat / n);
printf("\nAverage Response Time = %.2f\n", avg_rt / n);

    return 0; }
```

Output

```
C:\Users\student\Downloads\

Enter number of processes: 5
Enter Arrival Time, Burst Time and Priority (lower value = higher priority):
P1 AT: 0
P1 BT: 3
P1 Priority: 5
P2 AT: 2
P2 BT: 2
P2 Priority: 3
P3 AT: 3
P3 BT: 4
P3 Priority: 2
P4 AT: 4
P4 BT: 4
P4 Priority: 4
P5 AT: 6
P5 BT: 1
P5 Priority: 1

PID     AT      BT      Priority        FT      CT      TAT     WT      RT
P1      0       3       5               0       14      14      11      0
P2      2       2       3               2       9       7       5       0
P3      3       4       2               3       8       5       1       0
P4      4       4       4               9       13      9       5       5
P5      6       1       1               6       7       1       0       0

Average Waiting Time = 4.40
Average Turnaround Time = 7.20
Average Response Time = 1.00

Process returned 0 (0x0)   execution time : 83.067 s
Press any key to continue.
```